

FINAL PROJECT REPORT: Robotic Bouncing Leg

ME 239 Robotic Locomotion

Group 25: Jason Abi Chebli, Andrew Tai, Samantha Friedman, Carlos Goni Gil

December 9th, 2025

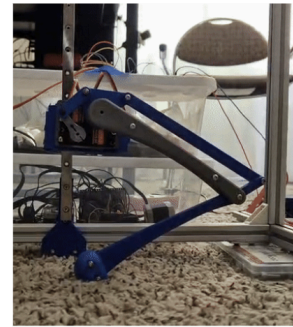
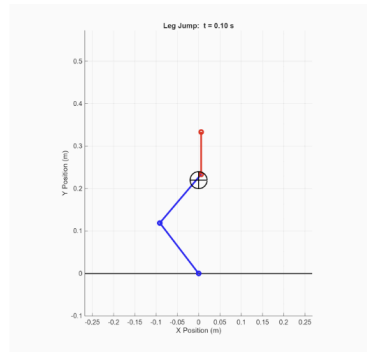
The following report is an adapted slide deck from the final presentation given on 12/03/2025.

Bouncing Leg



Team 25

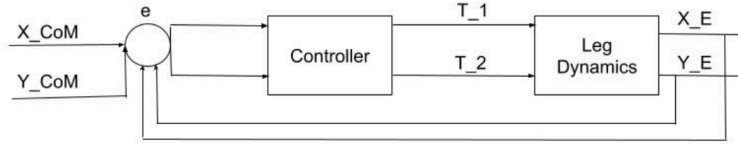
Jason Abi Chebli
Andrew Tai
Samantha Friedman
Carlos Goni Gil



Team 25's goal was to simulate a single virtual robotic leg doing a vertical jump. The virtual system has three degrees of freedom, allowing movement in the horizontal and vertical directions but not into the page, as well as rotation along the axis normal to the planes of motion. In order to make the leg successfully jump and follow a desired path trajectory, we had to implement a virtual constraint into the Lagrangian Dynamics in accordance with the [Feedback Control of Dynamic Robot Locomotion](#) Textbook. While a physical prototype was built, the focus of the project was the virtual simulation, and due to time constraints, the same controller developed in the simulation was not implemented on the prototype. As such, a first pass on a very basic, 1 degree of freedom, jumping leg controller was implemented on the physical prototype with future work suggesting to implement the more complex controller discussed in this report. The following slides will walk through a breakdown of the system, Lagrangian dynamics, simulations displaying the results of the modeled controller, and a preliminary prototype of the system.

System Overview

Block Diagram



System COM

$$p_{COM} = \begin{bmatrix} x_{COM} \\ y_{COM} \end{bmatrix} = \begin{bmatrix} \frac{m_1 x_{COM1} + m_2 x_{COM2} + m_3 x_{COM3}}{m_1 + m_2 + m_3} \\ \frac{m_1 y_{COM1} + m_2 y_{COM2} + m_3 y_{COM3}}{m_1 + m_2 + m_3} \end{bmatrix}$$

Foot Position

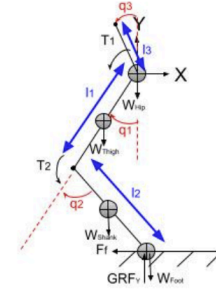
$$p_{foot} = \begin{bmatrix} x - l_1 \sin(q_1) + l_2 \sin(q_2 - q_1) \\ y - l_1 \cos(q_1) - l_2 \cos(q_2 - q_1) \end{bmatrix}$$

In order to successfully make a virtual leg jump, we need to ensure that it follows a trajectory of standing, squatting, and hyperextending to allow for the leg to get enough inertia moment to take off. To achieve this, we needed to control the horizontal (x) and vertical (y) position of the center of mass (COM) of the system, which can be determined using Forward Kinematics (FK). As such, our overall system can be modelled as the block diagram seen above. Using forward kinematics, we can know the true position of the COM at any time, and how it differs from our desired COM position required to make the leg jump and stabilize. In doing so, we can implement a virtual constraint which enforces a PD controller that will drive the actual COM position of the leg to the desired COM position by determining what the two motor torques should be. In other terms, the controller takes the position error, and controls the motor torques which then are fed into the “plant” (“leg dynamics” on the block diagram).

As seen in the free body diagram, our system consists of 3-links, the thigh, the shank and the torso. Each link has its own COM, inertia, and angle. The two revolute joints are actuated, one situated on the hip, and one on the knee. The position of the center of mass of each link, and positions of the start and ends of the links can all be determined using FK. Please note that the motors were placed on the hip joint to position the center of mass closer to the hip, and therefore increase the moment of inertia of the system. Torques are drawn such that both joints have the same positive convention.

In order to make the leg jump, Forward Kinematics (FK) is essential to know the position of the COM of each link, as well as its end and start points. Below are all the essential position expressions determined using FK. The forward kinematics for the position of different components of the system are defined in Cartesian coordinates. They are written in terms of the generalized coordinates (x, y, q₁, q₂, q₃). These position equations will be used to compute the Jacobians, center of mass equations, and constraint equations throughout the derivation. The expressions used can be seen below:

Free Body Diagram



Position of Knee (Forward Kinematics)

$$p_{knee} = \begin{bmatrix} x_{knee} \\ y_{knee} \end{bmatrix} = \begin{bmatrix} x - l_1 \sin(q_1) \\ y - l_1 \cos(q_1) \end{bmatrix}$$

Position of Foot (Forward Kinematics)

$$p_{foot} = \begin{bmatrix} x_{foot} \\ y_{foot} \end{bmatrix} = \begin{bmatrix} x - l_1 \sin(q_1) + l_2 \sin(q_2 - q_1) \\ y - l_1 \cos(q_1) - l_2 \cos(q_2 - q_1) \end{bmatrix}$$

Position of Torso (Forward Kinematics)

$$p_{torso} = \begin{bmatrix} x_{torso} \\ y_{torso} \end{bmatrix} = \begin{bmatrix} x - l_3 \sin(q_3) \\ y + l_3 \cos(q_3) \end{bmatrix}$$

Position of COM of Link 1 (Hip to Knee)

$$p_1 = \begin{bmatrix} x_{COM1} \\ y_{COM1} \end{bmatrix} = \begin{bmatrix} x - d_1 \sin(q_1) \\ y - d_1 \cos(q_1) \end{bmatrix}$$

Position of COM of Link 2 (Knee to Foot)

$$p_2 = \begin{bmatrix} x_{COM2} \\ y_{COM2} \end{bmatrix} = \begin{bmatrix} x - l_1 \sin(q_1) + d_2 \sin(q_2 - q_1) \\ y - l_1 \cos(q_1) - d_2 \cos(q_2 - q_1) \end{bmatrix}$$

Defining the system

General Lagrangian Equation:

$$D(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = B(q)u + J(q)\lambda$$

Coriolis Matrix
Input Matrix

Mass/Inertia Matrix
Gravity Vector
Constraint Matrix

Generalized Coordinates:

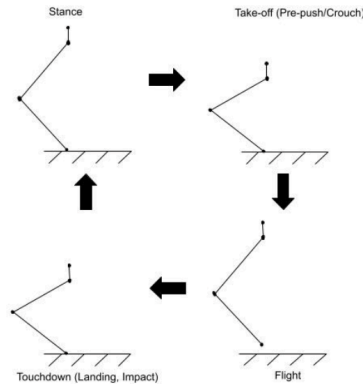
$$(q, \dot{q}) = \left\{ \begin{bmatrix} x \\ y \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}, \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix} \right\}$$

There are 4 stages we need to model:

1. Stance
2. Take-Off
3. During Flight
4. During Touchdown (impact mapping)

We want to find:

- Torque before take-off
- Torque during take-off
- Torque while in the air
- Torque on landing



Variable Definitions:

u : joint torques
 λ : ground reaction forces

Textbook: [Feedback Control of Dynamic Robot Locomotion](#)

We use the general Lagrangian equation which includes the mass/inertia matrix (D (also sometimes written as M)), the coriolis matrix (C), gravity vector (G), input matrix (B), and constraint matrix (J), to evaluate our system. The constraint matrix (J) will be useful to enforce virtual constraints to drive the COM of the system to a desired position. We took inspiration from the [Feedback Control of Dynamic Robot Locomotion](#) Textbook.

In the Lagrangian equation, u is the joint torque vector, λ is the ground reaction force vector, q is the generalized coordinates, $\dot{q}$ is the velocity of those coordinates, and $\ddot{q}$ is the acceleration of those coordinates. As mentioned earlier, the generalized coordinates include the x and y coordinates of the hip joint, and the three angles of the three links.

When modelling the hybrid dynamics of a virtual jumping leg, there are two main stages, stance and flight, and two transition stages, take-off and touchdown. The Lagrange equation is evaluated differently during each main stage. The figure in the center of the page illustrates the four stages of the system, noting that in all stages, the x -component of the COM is directly above the foot for balance and stability. Ultimately, we want to determine the torque during these stages required to drive the COM to the desired position.

Dynamics

During Stance:

We can rewrite the dynamics equation in this form:

Note on notation: we will use Lie Derivative notation

$$\dot{x} = \begin{bmatrix} \dot{q} \\ \dot{q} \end{bmatrix} = \underbrace{\begin{bmatrix} 0 \\ D^{-1}\{-C\dot{q} - G\} \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} 0 \\ D^{-1}B \end{bmatrix}}_{g_1(x)} u + \underbrace{\begin{bmatrix} 0 \\ D^{-1}J \end{bmatrix}}_{g_2(x)} \lambda$$

$$\underbrace{L_f h(x)}_{\text{Lie Derivative h along f}} = \frac{\partial h}{\partial x} f(x)$$

We want to enforce a virtual (holonomic) constraint that the COM x-position is above the position of the foot and that the COM y-position is at a desired position.

$$y = h(q) = h_0(q) - h_d(\theta(q))$$

In our case:

$$y = h = \begin{bmatrix} p_{COM_x} - p_{foot_x} \\ p_{COM_y} - p_{COM_y_d} \end{bmatrix} \begin{matrix} \% \text{ Balance} \\ \% \text{ Posture, } p_{COM_y_d} : \text{desired y COM} \end{matrix}$$

We want to find $u(t)$ such that,

1. If $y(0) = 0$ then $y(t) = 0$ for $t \geq 0$ (invariance)
2. If $y(0) \neq 0$ then $y(t) \rightarrow 0$ as $t \rightarrow \infty$ (asymptotic attractivity)

Textbook: [Feedback Control of Dynamic Robot Locomotion](#)

Starting with the stance stage, we can reorganize the Lagrangian dynamics to create a variable $\dot{x}$. Each component can be grouped into three sections, $f(x)$, $g_1(x)$ and $g_2(x)$ as seen in the figure above. Note that later on we will use the Lie Derivative notation to make the math easier to read.

As mentioned previously, to make the system jump we need to drive the x and y COM position of the entire system to some desired value. To do so, as described in the [Feedback Control of Dynamic Robot Locomotion](#) Textbook, we will want to enforce two holonomic constraints: one for the x-position of the COM, and one for the y-position of the COM. We first want to define a holonomic constraint that enforces the x-position of the systems COM is directly above the x-position of the foot. In other words, the foot and COM x-coordinate, will lay on the same imaginary vertical line. This allows the whole system to be stable. Without this constraint, the leg will just fall over due to slippage, and/or tipping. The second holonomic constraint enforces that the center of mass in the y-direction is at a desired position which we can define. This defines the “pose” of the leg, and is necessary in order to make the system follow a trajectory starting in a neutral position, squatting, and then extending to takeoff.

Overall, the holonomic virtual constraint is simply the difference between the current position and the desired position of the COM (it can be thought of as the ‘error’). Therefore, we want to find torque (written as ‘ u ’ in the dynamics equation) values for the hip and the knee, such that if the error (the holonomic constraint) is already zero, that it stays zero (invariant), and that if it is not at the current desired position, that it drives the system to it (asymptotic attractivity). In order for this to work, we need to find a relationship between the holonomic constraint, and the torque (u), which can be done by taking the derivative of the holonomic constraint twice as seen below.

Note of convention: the holonomic constraint is often written as “y” and “h”, as defined in the next slides

Dynamics

Taking the first time derivative of the virtual constraint:

$$\begin{aligned}\dot{y} &= \frac{dh}{dx} \frac{dx}{dt} \\ &= \frac{dh}{dx} \dot{x} \\ &= \left[\frac{\partial h}{\partial q} \quad \frac{\partial h}{\partial \dot{q}} \right] \left\{ \begin{bmatrix} \dot{q} \\ -C\dot{q} - G \end{bmatrix} + \begin{bmatrix} 0 \\ D^{-1}B \end{bmatrix} u + \begin{bmatrix} 0 \\ D^{-1}J \end{bmatrix} \lambda \right\}, \quad \frac{\partial h}{\partial \dot{q}} = 0 \\ &= \left[\frac{\partial h}{\partial q} \quad 0 \right] \left\{ \begin{bmatrix} \dot{q} \\ -C\dot{q} - G \end{bmatrix} + \begin{bmatrix} 0 \\ D^{-1}B \end{bmatrix} u + \begin{bmatrix} 0 \\ D^{-1}J \end{bmatrix} \lambda \right\} \\ \dot{y} &= \frac{\partial h}{\partial q} \dot{q}\end{aligned}$$

Using the Lie-Derivative Notation:

$$L_f h = \frac{\partial h}{\partial q} \dot{q}, \quad L_g h = 0$$

Taking the second time derivative of the virtual constraint:

$$\begin{aligned}\ddot{y} &= \frac{d}{dt} \left(\frac{dh}{dx} \dot{x} \right) = \frac{d}{dt} \left(\frac{\partial h}{\partial q} \dot{q} \right) = \frac{dL_f h}{dt} = \frac{dL_f h}{dx} \frac{dx}{dt} = \frac{dL_f h}{dx} \dot{x} \\ &= \left[\frac{\partial L_f h}{\partial q} \quad \frac{\partial L_f h}{\partial \dot{q}} \right] \left\{ \begin{bmatrix} \dot{q} \\ -C\dot{q} - G \end{bmatrix} + \begin{bmatrix} 0 \\ D^{-1}B \end{bmatrix} u + \begin{bmatrix} 0 \\ D^{-1}J \end{bmatrix} \lambda \right\} \\ &= \left[\frac{\partial}{\partial q} \left(\frac{\partial h}{\partial q} \dot{q} \right) \quad \frac{\partial}{\partial \dot{q}} \left(\frac{\partial h}{\partial q} \dot{q} \right) \right] \left\{ \begin{bmatrix} \dot{q} \\ -C\dot{q} - G \end{bmatrix} + \begin{bmatrix} 0 \\ D^{-1}B \end{bmatrix} u + \begin{bmatrix} 0 \\ D^{-1}J \end{bmatrix} \lambda \right\} \\ &= \left[\frac{\partial}{\partial q} \left(\frac{\partial h}{\partial q} \dot{q} \right) \quad \frac{\partial h}{\partial q} \right] \left\{ \begin{bmatrix} \dot{q} \\ -C\dot{q} - G \end{bmatrix} + \begin{bmatrix} 0 \\ D^{-1}B \end{bmatrix} u + \begin{bmatrix} 0 \\ D^{-1}J \end{bmatrix} \lambda \right\}\end{aligned}$$

Using the Lie-Derivative Notation:

$$\begin{aligned}\ddot{y} &= \frac{d}{dx} L_f h f(x) + \frac{d}{dx} L_f h g_1(x) u + \frac{d}{dx} L_f h g_2(x) \lambda \\ \ddot{y} &= L_f^2 h + L_{g_1} L_f h u + L_{g_2} L_f h \lambda\end{aligned}$$

Textbook: [Feedback Control of Dynamic Robot Locomotion](#)

In order to implement the feedback linearization controller, we need to get an expression between the constraints we want to enforce in terms of input u. Taking the first derivative of the constraint function y, we can apply the chain rule, then substituting the definition of $\dot{x}$, to get a final value. Since the definition of “holonomic” constraint is not dependent on the velocity, meaning $\partial h / \partial \dot{q} = 0$, the final expression we are left with is $L_f h$ (in Lie Derivative notation) showing that after the first time derivative we still do not have a relationship between the constraints and the torques (u). Therefore, we take the second time derivative of the holonomic constraint, using what we got from the first time derivative and the chain rule again. Substituting in values, we can get an expression between the acceleration of the holonomic constraint, the torques (u), and the reaction forces (λ). After simplifying the expression to Lie-Derivative notation, we obtain the final expression for the second derivative where:

- $L_f^2 h$: represents the natural system evolution without controls.
- $L_{g_1} L_f h$: show how the joint torques affect the output acceleration.
- $L_{g_2} L_f h$: represents the ground reaction forces effect on the output acceleration.

Dynamics

During Stance:

We want, $y(t) \rightarrow 0$ as $t \rightarrow \infty$, and so, this is possible if we implement a PD controller such that $\ddot{y} = v$ where,

$$v = -(K_p y - K_d \dot{y}), \quad K_p > 0 \text{ and } K_d > 0$$

Now, we need to determine the three unknowns, $\ddot{q}$, u , λ , and three equations,

$$\begin{cases} D(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = B(q)u + J_{st}(q)\lambda \\ \ddot{y} = L_f^2 h + L_{g_1} L_f h u + L_{g_2} L_f h \lambda = v \\ J_{st} \ddot{q} + \dot{J}_{st} \dot{q} = 0 \end{cases} \longrightarrow \begin{bmatrix} D & -B & -J_{st} \\ 0 & L_{g_1} L_f h & L_{g_2} L_f h \\ J_{st} & 0 & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ u \\ \lambda \end{bmatrix} = \begin{bmatrix} -C\dot{q} - G \\ v - L_f^2 h \\ -\dot{J}_{st} \dot{q} \end{bmatrix}$$

We can solve simultaneously

Textbook: [Feedback Control of Dynamic Robot Locomotion](#)

We now need to implement the controller and solve for the required torques (u) and the ground reaction forces (λ) during stance to satisfy the virtual holonomic constraints. To do so, we implement a PD controller to drive $\ddot{y}$ to zero, which drives the x-position and y-position of the COM to be any desired posture position. The Lie Derivative parts allow us to linearize this normally non-linear system.

As seen above, to implement this controller, we chose v , equal to the holonomic virtual constraint acceleration, $\ddot{y}$, calculated above to be a PD controller. K_p is greater than 0 since it represents the proportional gain matrix, and K_d is the derivative gain matrix, also greater than 0.

The matrix shown on the slide has this expression for $\ddot{y}$ (second row of the matrix), the expression for the Lagrangian equation (first row in matrix), and we can define the kinematic control expression, ensuring that the foot is stationary during stance, (third row in matrix). We have three unknowns, so we can then solve this system of equations to get the generalized coordinate accelerations, the ground reaction forces experienced (λ), and the motor torques required (u) in order to drive the system's COM to a desired position. In MATLAB this will be solved at every time step using ODE45. With this, we've solved the hardest part for the first main stage of the jump.

Dynamics

During Flight (no contact, $\lambda=0$):

$$D(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = B(q)u$$

During Take-off:

$$\begin{bmatrix} D(q) & -J_{st}(q)^T \\ J_{st}(q) & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ F_{st} \end{bmatrix} = \begin{bmatrix} B(q)u - C(q, \dot{q})\dot{q} - G(q) \\ -J_{st}(q, \dot{q})\dot{q} \end{bmatrix}$$

During Touchdown (impact mapping):

$$\begin{aligned} D(q)\dot{q}^+ - D(q)\dot{q}^- &= J_{st}^T(q)F_{imp} \\ \Rightarrow \begin{bmatrix} J_{st}(q) & 0 \\ D(q) & -J_{st}^T(q) \end{bmatrix} \begin{bmatrix} \dot{q}^+ \\ F_{imp} \end{bmatrix} &= \begin{bmatrix} 0 \\ D(q)\dot{q}^- \end{bmatrix} \end{aligned}$$

Textbook: [Feedback Control of Dynamic Robot Locomotion](#)

During flight, the second main stage of the jump, the system does not have any ground contact. Therefore, we can then eliminate the ground virtual constraint forces and simplify the system considerably. The COM then follows a ballistic trajectory solely affected by gravity. Although the COM trajectory is fixed, we can try to reposition the leg's joints to prepare for landing.

Now we will discuss the transition stages, take-off and touchdown. Mathematically, take-off can be represented using the coupled dynamics-constraint system seen above. Take-off corresponds to the moment when the stance force satisfies $F_{st}=0$. In practice, however, we do not need a special take-off model because take-off is not an impulsive event - the system velocities remain continuous. As soon as the ground reaction force (λ) computed during stance becomes zero, the foot is no longer supporting the robot and the stance constraint is removed. The system naturally transitions into the flight dynamics without requiring an impact map or momentum jump conditions. Only touchdown needs explicit modeling, since it introduces a velocity discontinuity and impulsive ground forces.

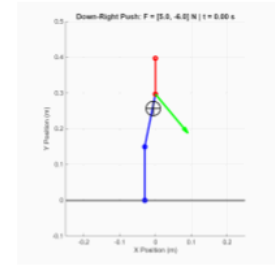
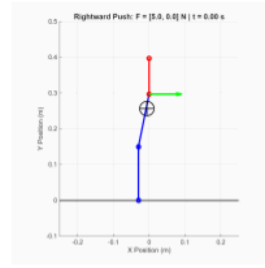
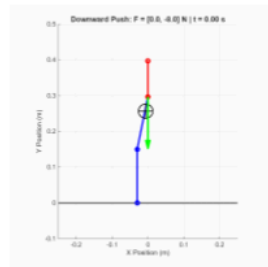
During touchdown, the foot makes contact with the ground with a non-zero velocity. In order to model this discontinuity of the system. An expression for the conservation of momentum is defined for the system where:

- $\dot{q}^-$: Velocities immediately before impact
- $\dot{q}^+$: Velocities immediately after impact
- F_{imp} : Impulsive ground reaction force

To complete the system, we define the instantaneous velocity of the foot during touchdown to be zero instantaneously.

Stability Conditions

Animation of Leg:
Vertical Perturbation
Horizontal Perturbation
Combination Perturbation

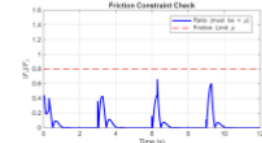
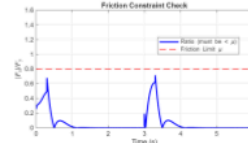
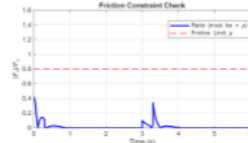


Condition (i): No Lift Off
Vertical Force Check ($F_y > 0$)



Condition (ii): No Slipping
Friction Ratio Check ($|F_x| / F_y < \mu$)

Assume $\mu = 0.8$



To validate the effectiveness of the virtual constraint, we first evaluated whether the system could maintain stability under significant perturbations. This was important not only to verify that the virtual constraint behaves as expected, but also because, even if the foot begins in a stable configuration, we do not have control of it during the flight phase. As a result, the foot may not land perfectly aligned, and the stance controller must be capable of re-stabilizing the system.

This slide highlights the system's ability to remain stable under large vertical, horizontal, and combined perturbations (shown from left to right). The virtual constraint was defined such that the COM x-position tracks the foot's x-position, and the COM y-position is driven to 0.2m, which is an arbitrary but suitable stance height for maintaining stability. During these perturbation tests, we evaluated two conditions: no lift-off and no slipping.

For the no lift-off condition, the vertical ground reaction force must remain positive. The simulation results confirm that the leg does not experience lift-off under any of the tested perturbations.

For the no-slipping condition, assuming a friction coefficient of 0.8, we checked that $|F_x|/F_y < \mu$. Satisfying this inequality ensures that the required tangential force remains below the maximum available static friction. All three perturbation scenarios met this condition, demonstrating that the system remains stationary and does not slip. Naturally, if the perturbations were increased beyond the tested values, slipping would eventually occur; however, in practice the foot is not expected to encounter such extreme disturbances for just vertical jumping. These tests demonstrate that the system can tolerate reasonable deviations in the COM position and still successfully drive the leg back to the desired configuration quickly.

Overall, these results confirm the stability of the leg during stance and validate the implementation of the virtual constraint.

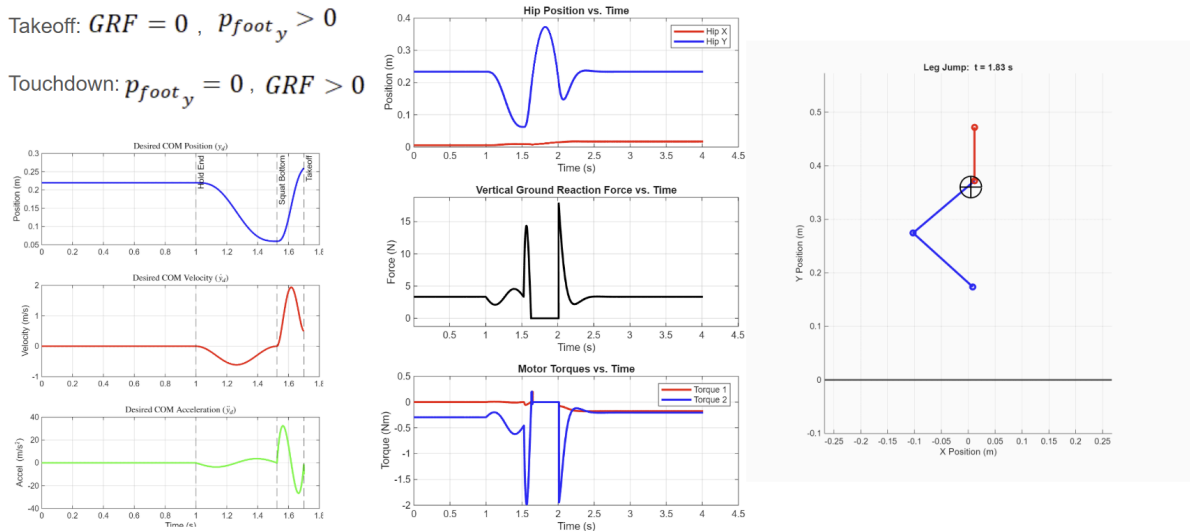
Note: ZMP was not used because it is limited to flat-footed, quasi-static walking, whereas virtual

constraints provide a more general and effective framework for stabilizing our point-foot system across stance and flight phases. To reiterate, the goal of this project was to make a jumping leg.

Jumping Conditions

Takeoff: $GRF = 0$, $p_{foot_y} > 0$

Touchdown: $p_{foot_y} = 0$, $GRF > 0$



Now that we know our virtual constraints controller is working, the next step is to try to make it jump. We already know we need to determine the y-trajectory of the COM y-position required to generate enough upwards acceleration to takeoff. This was done in the left three plots. As can be seen, the starting position is 0.22m in stance, the leg then squats down to 0.06m before quickly accelerating to above 0.25m where it takes off. This can be seen in the top plot on the left. To ensure this actually resulted in takeoff during the ascent, we had to calculate the velocity so that we can then calculate the acceleration of the y-COM and ensure that during the extension upwards, the acceleration is greater than gravity (9.81m/s^2). If so, the foot will take off. After finding this optimal trajectory path, we now have the path that we will drive the COM y-position of the system to in our constraints.

In MATLAB the event will follow this trajectory until the takeoff event is triggered. We know the foot has taken off if the ground reaction force (λ) is equal to 0N. The point in time where the GRF first equals 0N is known as the takeoff event, signaling the beginning of the flight stage of the jump. This then results in us using the flight stage dynamics equation instead of the stance stage dynamics equations. During flight the only force acting on the leg is gravity and so eventually it will touch the ground. We know it touches the ground when the y-position of the foot becomes 0, meaning it has reached contact with the ground. As soon as this happens, the stance stage activates again, this time focusing on stabilizing it, making it ready for another jump.

While the three figures on the left show the position, velocity and acceleration trajectory during stance, the actual x and y position that the jumping leg experienced, the ground reaction force and motor torques can be seen in the center three figures.

In the images in the center of the slide, we show the hip position vs time for a single jump. It shows the initial stance phase where there is no position change, the crouch to prepare for takeoff shown by the decrease in Y hip position, flight shown by the area from about 1.5-2s, followed by touchdown before returning to stance phase. You can even see that during touchdown the leg absorbs a bit of shock before returning back to its stable position. When

comparing the top center graph with the ones below it, you can see that during flight, the ground reaction force is 0N and that the ground reaction force in the Y direction is slightly higher at touchdown than at takeoff.

The image in the bottom center of the slide shows the motor torques required to control each joint. You can see that the torque required for both motors during flight is 0Nm. The highest torque required of either motor is for the hip joint and is about 2Nm and is needed during takeoff and touchdown. Finding this value was important because it allowed us to spec servo motors that we used for our preliminary prototype that's shown on the next slide. On the right-hand side of the slide, you can see an animation of the controller creating a jump of the leg. This is what we would expect to see in real life if we built this exact system as it was modeled.

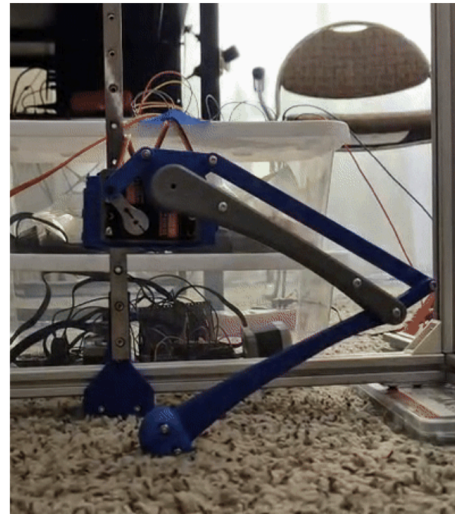
Prototype

A first-version physical prototype was developed.

A jumping characteristic was achieved using a simplified joint angle controller that drives the leg to set values.

If we had more time, and money, we would:

- Redesign the test-rig and mounting apparatus
- Implement an IMU and do inverse/forward kinematics of the leg
- Use Brushless motors (to get force feedback for when we touch the ground)
- Transition the stabilizing controller and jumping controller implemented in MATLAB to C++ (arduino)



Although the main goal of the project was to virtually simulate the robotic bouncing leg, the team was interested in trying to apply the dynamic model to a real prototype. It is important to note that due to time constraints and our priority being the mathematical model for the controller, we were unable to fully develop a prototype that matches the complex model we have shown throughout the report.

The prototype shown on the right hand side of the screen was made by 3D printing the leg. This allowed the team to use real mass values, moments of inertia, and link center of mass locations in the mathematical model. Other components used for the prototype include 20mm t-slot framing and a 300mm linear rail that the leg jumps up and down along. The current prototype uses 25 kg-cm servo motors.

The jumping motion shown in the video was achieved using a simplified controller that drove the hip and knee joints to set angles. Thus, the prototype does not give a fully accurate physical representation of the modeled controller discussed throughout the presentation.

If we had more time, the goal would be to fully integrate the stabilizing and jumping controllers with an Arduino microcontroller. To do this, we would need to implement an IMU that would give us accurate positional/rotational information so that we could do inverse kinematics and forward kinematics of the leg. Something else we would like to implement to get more feedback from the system would be to use brushless motors. This would allow us to get force feedback to better detect stance and flight phases.

All the weight is at the hip because you want all the mass as high up so that you reduce the inertia of each leg which means the legs can move faster.

To reiterate, this prototype was not the group's focus for the semester. The goal would be to continue building off this prototype to fully integrate the developed controllers.

Questions?



<https://github.com/jabichebli/Jumping-Leg>

Feel free to scan the QR Code or follow the link to our github (<https://github.com/jabichebli/Jumping-Leg>), and try out the controller.

References:

Westervelt, E. R., Grizzle, J. W., Chevallereau, C., Choi, J. H., & Morris, B. (2007, May 15). Feedback Control of Dynamic Bipedal Robot Locomotion. https://web.eecs.umich.edu/~grizzle/papers/Westervelt_biped_control_book_15_May_2007.pdf